

KI-gestützte Testgenerierung mit PHPUnit in TYPO3

(Evaluation, Qualitätsanalyse, Grenzen, Workflow)

Autor: Sinian Zhang

Ort, Datum: Rombach, 23. Juli 2026

Geheimhaltung: Vertraulich (nur innerhalb der FHNW einsehbar)

Betreuer/Dozierende: Samuel Fricker, Dan Burns

Ehrenwörtliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der im Literaturverzeichnis angegebenen Quellen und Hilfsmittel angefertigt habe.

Die wörtlich oder inhaltlich den im Literaturverzeichnis aufgeführten Quellen und Hilfsmitteln entnommenen Stellen sind in der Arbeit als Zitat bzw. Paraphrase kenntlich gemacht.

Ort, Datum

Unterschrift

Management Summary / Abstract

(die Arbeit soll 40-60 Seiten haben, ohne Anhänge)

Inhaltsverzeichnis

1Einleitung.....	6
1.1Problembeschreibung	7
1.2Organisatorische Einbettung.....	8
1.3KI-bezogene Unternehmens-, Projekt-, und Lernziele.....	9
1.4Stakeholder und Kontextanalyse.....	10
2Planung.....	11
2.1Wertstromanalyse.....	12
2.2Geplanter KI-Einsatz.....	13
2.3Kriterien zur Bewertung und erwartete Benefits.....	14
2.4Vorgehen zur Einführung und Validierung.....	14
2.5Bekannte Chancen und Risiken aus Praxis und Forschung.....	15
3Umsetzung.....	16
3.1Tooling und Infrastruktur.....	17
3.2Entwickltes Resultat: Claude Code CLI Plugin.....	18
3.3Nutzung von Plugin.....	18
3.4Beispielbasierte Demonstration.....	18
3.5Pilotnutzung.....	20
3.6Messung und Beobachtung der Benefits und Effekte.....	21
4Diskussion.....	22
4.1Erreichte Ergebnisse und Beitrag der Arbeit.....	23
4.2Bezug zu bestehenden Erfahrungen und Studien.....	24
4.3Empfehlung, Innovation und Lernen.....	25

1 Einleitung

(Zusammenfassung)

1.1 Problembeschreibung

Automatisierte Tests gelten als Grundpfeiler moderner Softwareentwicklung. Aus meiner langjährigen Erfahrung in verschiedenen Webagenturen mit TYPO3-Projekten zeigt sich jedoch: In der Praxis werden sie häufig weggelassen — nicht weil Entwicklerinnen und Entwickler ihren Nutzen nicht kennen, sondern weil der Aufwand im Projektalltag mit engem Zeitbudget schlicht zu hoch ist.

Genau diesen Engpass adressiert die vorliegende Arbeit. LLMs können PHP-Code analysieren und daraus Testcode generieren. Die zentrale Frage ist: Sind diese Tests praxistauglich — laufen sie durch, decken sie den Code sinnvoll ab, und erkennen sie echte Grenzwerte?

Zugleich ist mir die Arbeit eine Gelegenheit, sich intensiv mit einem modernen Toolset auseinanderzusetzen: Claude Code, PHPUnit mit dem TYPO3 Testing Framework, Infection für Mutationstests und PHPStan für die statische Analyse, etc.

Noch wichtig zu erwähnen: PHP-Klassen enthalten oft mehr Information als nur den ausführbaren Code — zum Beispiel in PHPDoc- oder Inline-Kommentaren, die erklären, welche Werte eine Methode erwartet, was sie zurückgibt und warum sie so implementiert wurde. Werden diese Informationen als zusätzlicher Input an das LLM übergeben, lassen sich sicherlich gezielter und qualitativ bessere Tests erzeugen. In dieser Arbeit werden Tests bewusst ohne solche Kommentare als Hinweis generiert, was zeigt, was das LLM allein aus dem Quellcode ableiten kann.

Untersucht wird dies anhand von fünf Klassen aus einer produktiven TYPO3-Extension. Die Klassen sind bewusst unterschiedlich gewählt — von einfacher Dummy-Logik über reine PHP-Logik bis hin zu komplexem TYPO3-abhängigem Code mit echtem Mocking-Bedarf. Die Ergebnisse werden quantitativ anhand von vier Kennzahlen gemessen: Erstellungszeit, Methodenabdeckung, PHPStan-Fehler und Mutation Score. Diese bilden zusammen die Grundlage für eine Einschätzung des Praxisnutzens KI-gestützter Testgenerierung.

1.2 Organisatorische Einbettung

Unternehmenskontext

Die vorliegende Arbeit entsteht im Rahmen meiner beruflichen Tätigkeit beim ehemaligen Arbeitgeber Hausformat GmbH, einer Agentur mit Fokus auf TYPO3-Webentwicklung. Im Tagesgeschäft sind Entwicklerinnen und Entwickler für die Implementierung und Pflege von TYPO3-Extensions zuständig. Automatisierte Tests sind zwar bekannt, wie oben schon erwähnt, werden jedoch aufgrund des hohen manuellen Aufwands selten konsequent umgesetzt.

Den direkten Anlass für diese Arbeit liefert eine konkrete Aufgabe: Für die Extension EXT:hf-view-helpers — eine intern entwickelte ViewHelper-Sammlung, die keine kundenspezifische Anwendung implementiert daher auch keine kundenspezifischen Daten enthält, die eher als Framework bei zahlreichen Kundenprojekten im Einsatz ist — sollte eine PHPUnit-Testumgebung aufgesetzt und nachträglich für einige zentrale Klassen Unit-Tests geschrieben werden. Die Extension hat viele Versionen, die betroffene ist eine Version mit der TYPO3-Version V14 kompatibel ist.

Diese Arbeit setzt genau an diesem Punkt an: Sie untersucht, ob ein LLM diesen Aufwand so weit reduzieren kann, dass Unit-Tests zum Standardbestandteil jedes Entwicklungszyklus werden.

Projektkontext und Demo-Extension

Als technische Basis dient das TYPO3-Demoprojekt 'cas_ai_phpunit' (https://github.com/sinianzhang/cas_ai_phpunit), das lokal mit DDEV (Docker-basierte Entwicklungsumgebung) betrieben wird. Innerhalb dieses Projekts wird die Extension 'hf-view-helpers' als Testgegenstand verwendet. Sie bündelt fast 50 TYPO3 Fluid-ViewHelper-Klassen, die im Produktivbetrieb eingesetzt werden.

Um die Testbarkeit der Klassen systematisch zu erfassen, wurde im Rahmen dieser Arbeit ein eigenes Claude-Code-Plugin(CLI) entwickelt: 'typo3-test-audit'. Es enthält vier Skills, darunter der Skill 'test-audit-text' analysiert alle PHP-Klassen einer Extension und erstellt einen Überblick — wie viele Klassen existieren, welche sich für PHPUnit Unit-Tests eignen und welche funktionale Tests erfordern, jeweils mit kurzer Begründung. Die Auswahl der fünf Beispielsklassen für diese Arbeit basiert auf diesem Audit-Report.

Das Plugin ist nicht projektspezifisch und kann künftig bei beliebigen TYPO3-Projekten wiederverwendet werden — ein zusätzlicher Mehrwert/Benefit dieser Arbeit. Die Funktionsweise des Plugins wird in Abschnitt 3.1 erläutert.

1.3 KI-bezogene Unternehmens-, Projekt-, und Lernziele

Die Ziele dieser Arbeit lassen sich auf drei Ebenen betrachten.

Unternehmensziele (für ehemaligen auch zukünftigen Arbeitgeber)

Es steht im Vordergrund, dass automatisierte Tests im Alltag tatsächlich geschrieben werden — und nicht nur in der Theorie sinnvoll wären. Wie in Abschnitt 1.1 beschrieben, fehlt im Projektgeschäft schlicht die Zeit dafür.

Das Unternehmen erhofft sich von dieser Arbeit zwei konkrete Dinge für die Test-Klassen:

Weniger Aufwand: Wenn ein grosser Teil der Testerstellung durch KI übernommen (mit oder ohne Inline-Kommentar zur Generierung bestimmtes TestCases) werden kann, sinkt die Hürde, Tests überhaupt zu schreiben.

Weniger Fehler: Das senkt langfristig das Risiko, dass die UnitTestClass selbst fehlerhaft ist und dadurch ein falsches Sicherheitsgefühl entsteht. Hier wird in erster Linie auf Fehler beschränkt, die statisch erkennbar sind.

Persönliches Lernziele

Neben den beiden inhaltlichen Zielen ist mir die Arbeit auch persönlich wichtig, um mich als Entwickler weiterzuentwickeln. PHPUnit-Tests waren bislang nicht Teil meines Alltags — genau das macht diese Arbeit für mich zu einer guten Gelegenheit, mich intensiv und strukturiert mit dem Thema Testing auseinanderzusetzen: Wie schreibt man sinnvolle Testfälle, wie funktioniert Mocking von TYPO3-Abhängigkeiten, wie liest man einen Coverage-Report richtig, und was sagt ein Mutation Score tatsächlich aus? Gleichzeitig lerne ich den Umgang mit einem modernen Toolset, das über PHPUnit hinausgeht — insbesondere Claude Code als KI-gestützte Entwicklungsumgebung, PHPStan für die statische Analyse und Infection für Mutationstests. Dieses Wissen bleibt auch über die Arbeit hinaus nutzbar, sowohl für mich persönlich als auch für den Einsatz beim Arbeitgeber.

Projektziele (CAS-Arbeit)

Für die Arbeit selbst verfolge ich drei Hauptziele, die auch die Struktur der Auswertung in Kapitel 3 und 4 bestimmen:

Erstes Hauptziel — Ein wiederverwendbares Werkzeug: Das im Rahmen dieser Arbeit entwickelte Plugin typo3-test-audit (siehe Abschnitt 1.2 und 3.1) soll nicht nur für diese Arbeit, sondern auch danach im Tagesgeschäft nutzbar sein — für beliebige TYPO3-Extensions, nicht nur für hf-view-helpers.

Zweites Hauptziel — Machbarkeit zeigen: Ich will nachweisen, dass sich aus einer bestehenden PHP-Klasse mithilfe eines LLM automatisch eine lauffähige PHPUnit-Testklasse erzeugen lässt, ohne dass ich die Tests von Grund auf selbst schreiben muss. „Lauffähig“ heisst hier ganz konkret: Die Tests lassen sich mit PHPUnit ausführen, sie bestehen (grün) und sie prüfen sinnvolle Fälle statt nur oberflächlich zu bestehen.

Drittes Hauptziel — Fairer Vergleich mit Zahlen: Ich will nicht nur behaupten, dass KI-generierte Tests gut sind, sondern das mit Zahlen belegen. Dazu vergleiche ich für jede der fünf ausgewählten Klassen die von mir manuell geschriebenen Tests mit den KI-generierten Tests, anhand von vier klar messbaren Kriterien:

Erstellungszeit — wie lange dauert es, bis eine lauffähige, fehlerfreie Testklasse vorliegt?

Methodenabdeckung (Coverage) — wie viele der öffentlichen Methoden werden überhaupt von einem Test aufgerufen?

PHPStan-Fehler — wie sauber und typsicher ist der generierte Testcode?

Mutation Score — prüfen die Tests wirklich die Logik, oder würden sie auch bei einer fehlerhaften Implementierung noch grün bleiben?

1.4 Stakeholder und Kontextanalyse

An dieser Arbeit sind mehrere Parteien beteiligt oder zumindest indirekt betroffen, auch wenn nicht alle aktiv mitarbeiten. Die folgende Übersicht beschreibt für jede Partei, welches Interesse sie an der Arbeit hat und wie stark sie mitbestimmen kann, wie die Arbeit abläuft und welche Richtung sie nimmt.

Sinian Zhang (Autor und Entwickler) — Einfluss: hoch Ich schreibe die Arbeit und bin gleichzeitig der Entwickler, der die Tests erstellt und die KI-Ergebnisse prüft. Ich habe zwei Ziele: Erstens will ich im Projektalltag Zeit sparen, indem ein grosser Teil der Testerstellung automatisch läuft. Zweitens will ich im Rahmen des CAS solides Wissen zu PHPUnit-Testing sowie einige Tools aufbauen (siehe Lernziel in Abschnitt 1.3). Weil ich sowohl die Methodik festlege als auch die Messungen selbst mache, habe ich den grössten Einfluss auf die Arbeit.

Ehemaliger oder zukünftiger Arbeitgeber — Einfluss: mittel Die Firma profitiert wirtschaftlich von besserer Code-Qualität. Mein Arbeitgeber hat den ursprünglichen Anlass für das Thema geliefert (siehe Abschnitt 1.2) und liefert mit dem Demoprojekt den fachlichen Rahmen, mischt sich aber nicht in die wissenschaftliche Methodik oder einzelne Entscheidungen der Arbeit ein. Auch für einen zukünftigen Arbeitgeber ist die Arbeit relevant: Sowohl das Plugin typo3-test-audit als auch das dabei aufgebaute Fachwissen zu KI-gestützter Testgenerierung lassen sich in künftigen Projekten wiederverwenden.

TYPO3-Community — Einfluss: niedrig Für die breitere TYPO3-Community ist die Arbeit interessant. Da das Plugin typo3-test-audit wiederverwendbar ist (siehe Abschnitt 1.2), könnten die Ergebnisse auch anderen TYPO3-Entwicklerinnen und -Entwicklern nützen. Die Community ist aber nicht aktiv an der Arbeit beteiligt und hat deshalb keinen direkten Einfluss auf deren Verlauf.

FHNW / Dozent — Einfluss: hoch Aus akademischer Sicht steht der wissenschaftliche Beitrag der Arbeit im Vordergrund: eine nachvollziehbare Methodik, überprüfbare Hypothesen (siehe Abschnitt 2.3) und eine saubere, messbare Auswertung. Der Dozent bzw. die Dozentin begleitet die Arbeit fachlich, gibt Feedback zu Aufbau und Methodik und bewertet am Ende das Ergebnis. Damit ist der Einfluss auf Anforderungen und Qualitätsmassstäbe hoch, auch wenn ich die inhaltliche Umsetzung selbst mache.

2 Planung

(Zusammenfassung)

2.1 Wertstromanalyse

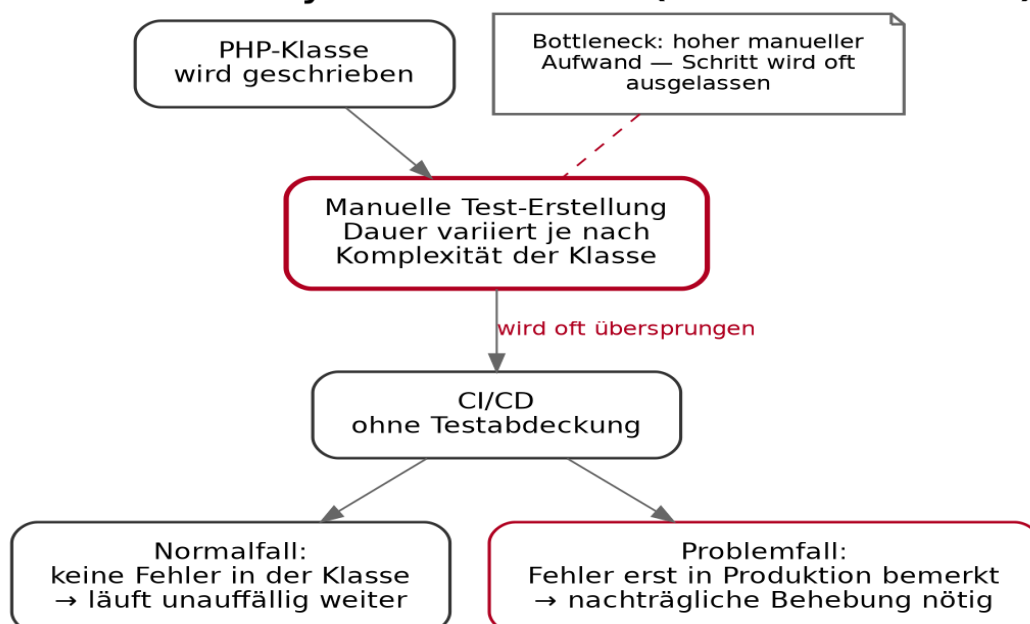
Die Wertstromanalyse (Value Stream Mapping, VSM) ist eine aus dem Lean Management stammende Methode, die den gesamten Arbeitsfluss „von der Idee bis in Produktion“ visualisiert und dabei Engpässe (Bottlenecks) sowie Wartezeiten zwischen einzelnen Prozessschritten sichtbar macht (<https://dora.dev/guides/value-stream-management/>). Im Kontext dieser Arbeit wird der Wertstrom auf den relevanten Ausschnitt eingegrenzt: von der fertig geschriebenen PHP-Klasse bis zur produktiv nutzbaren Testabdeckung.

Ist-Zustand:

Sobald eine PHP-Klasse fertig geschrieben ist, müsste als nächster Schritt die passende Testklasse manuell erstellt werden. Das dauert pro Klasse je nach Komplexität mehr (vgl. Messdaten Abschnitt 3.5). Weil dieser Aufwand im Projektalltag mit engem Zeitbudget kaum aufzubringen ist, wird dieser Schritt in der Praxis sehr häufig ganz übersprungen — also der Bottleneck sitzt hier also innerhalb eines einzelnen Schritts, der manuellen Testerstellung selbst (siehe H1, Abschnitt 2.3). Die Klasse landet dadurch ohne Testabdeckung in der CI/CD-Pipeline.

Dahinter steckt eine Unterscheidung, die auch das DORA-Modell zur Messung von Software-Delivery-Performance trifft: den Normalfall, bei dem eine Funktion ohne Probleme bis in die Produktion läuft, und den Problemfall, bei dem ein Fehler erst in Produktion bemerkt und dort nachträglich behoben werden muss. Fehlende Tests verlangsamen den Normalfall nicht direkt, machen aber den deutlich teureren Problemfall wahrscheinlicher.

Wertstromanalyse — Ist-Zustand (manueller Prozess)



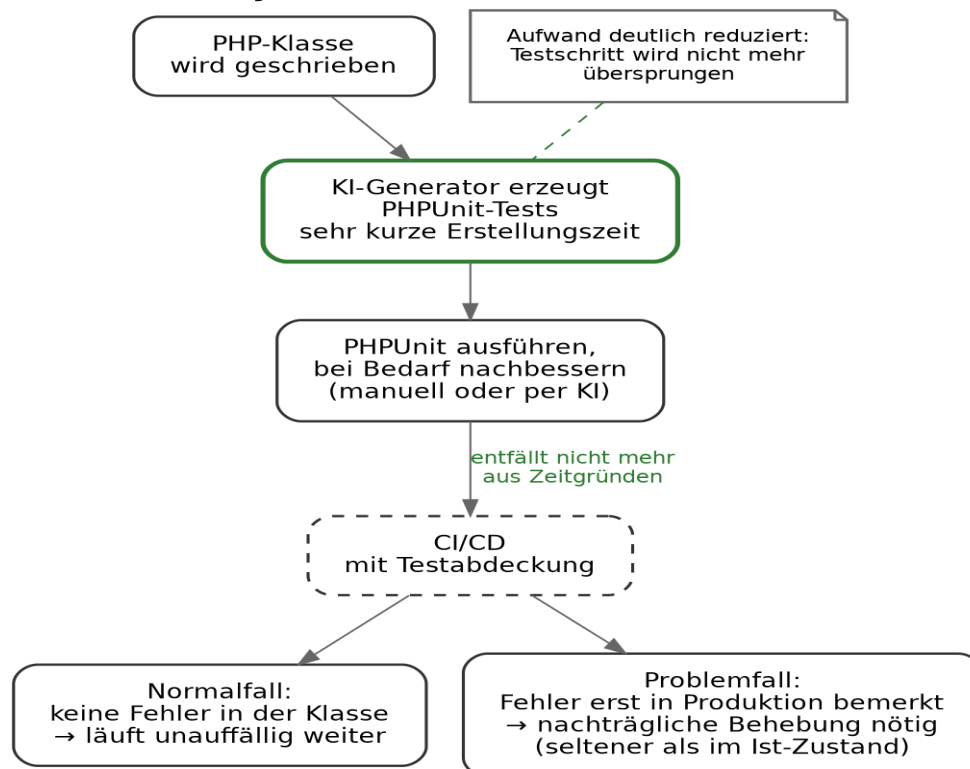
Soll-Zustand:

Sobald die PHP-Klasse fertig geschrieben ist, erzeugt der KI-Generator die passende Testklasse in sehr kurzer Zeit inklusive Korrektur (vgl. Messdaten Abschnitt 3.5). Anschliessend wird die Testklasse mit PHPUnit ausgeführt und bei Bedarf in wenigen Minuten von Hand (ggf. per KI) nachgebessert. Weil dieser Schritt kaum noch Zeit kostet, entfällt er nicht mehr aus Zeitgründen, und die Klasse gelangt mit Testabdeckung in die CI/CD-Pipeline. Fehler werden dadurch früher erkannt, und der aufwändige Recovery-Pfad — also die nachträgliche Fehlerbehebung in Produktion — wird seltener durchlaufen.

CI/CD-Integration ist in der Praxis Standardverfahren, wird in dieser Arbeit jedoch bewusst nicht umgesetzt, sondern nur als Zielpunkt des Wertstroms mitgeführt — um den Aufwand im Rahmen des CAS auf die Testgenerierung selbst zu fokussieren (vgl. Abgrenzung in Abschnitt 1.2).

Bezug zu DORA-Metriken-Theorie: Weil die Testerstellung schneller geht, wird auch die Zeit von der Codeänderung bis zur Auslieferung kürzer — das misst DORA mit der Kennzahl Lead Time for Changes. Ausserdem werden Fehler eher schon vor dem Deployment gefunden statt erst danach, wodurch weniger fehlerhafte Änderungen in Produktion landen — DORA misst das mit der Change Failure Rate. Die Wertstromanalyse zeigt also, warum die Zeitersparnis durch KI (Hypothese H1 in Abschnitt 2.3) für diese Arbeit wichtig ist.

Wertstromanalyse — Soll-Zustand (mit KI-Unterstützung)



2.2 Geplanter KI-Einsatz

Die vollständigen Abläufe lassen sich per das folgende Diagramm darstellen: also 3 Abschnitte.

ABSCHNITT 1: INPUT & VORBEREITUNG

Code-Analyse aller PHP-Klasse einer Extension (KI-CLI-Skill:test-audit-text)

- public Methoden identifizieren
- Abhängigkeiten prüfen (DI, GeneralUtility, etc.)
- Klassifizierung: Unit / Edge / Functional

(ENTWEDER)

Generierung von Tests für alle Klasse (KI-CLI-Skill:generate-unit-tests)

(ODER)

Generierung von Tests für eine Klasse

- Prompt erstellen, mit AAA-Pattern (Claude Code)
- Mit oder ohne PHPDoc/ Inline-Kommentare

↓↓↓

ABSCHNITT 2: AUSFÜHRUNG & VALIDIERUNG

(je nach Komplexität von zu testenden Klassen)

LLM (Claude Code) generiert und speichert eine oder alle PHP-Testklassen

Kontrolle, Fix, Nachverbesserung(manuell oder per KI) einer oder aller PHP-Testklassen

- PHPUnit ausführen und analysieren
- Code Coverage
- PHPstan (Error-Fix: manuell oder per KI)
- Mutationstest per infection (Verbesserung: manuell oder per KI-CLI-Skill:fix-unit-tests)
- ggf. Sonstige manuelle oder subjektive Analyse und Korrektur

↓↓↓

ABSCHNITT 3: BEWERTUNG & ERKENNTNISSE

(5 ausgewählte Klassen, nur Messmethodik dieser Arbeit — kein Bestandteil des wiederverwendbaren Plugin-Workflows)

Vergleich: Manuell vs. KI-generiert, Kriterien

- Laufen Tests durch? (ja / nein / nach Korrektur)
- Code Coverage Funktions- und Methodenabdeckung (%)
- Assertions sinnvoll? (Edge-Cases, Grenzwerte)
- Mocking korrekt? (TYPO3-Dependencies)
- PHPstan (Error-Fix per KI)
- Mutationstest (%) (infection)
- Zeitmessung, Assertions-Stufe identifizierung, etc.

Bemerkungen:

- Die Abschnitte 1 und 2 des obigen Ablaufs bilden den generischen Prozess, den z.B. das Plugin typo3-test-audit bei jeder beliebigen TYPO3-Extension durchläuft — unabhängig vom konkreten Projekt.
- Abschnitt 3 dagegen — der Vergleich mit manuell geschriebenen Tests — ist kein Bestandteil dieses wiederverwendbaren Ablaufs, sondern ausschliesslich die Messmethodik, mit der in dieser CAS-Arbeit die Praxistauglichkeit der KI-generierten Tests überprüft wird (siehe Abschnitt 3.5).
- Die konkrete Anwendung auf ausgewählte 5 Klassen (inkl. Begründung der Auswahl) ist bereits Inhalt von 2.4 "Vorgehen zur Einführung und Validierung" und wird in 3.2/3.4 mit echten Ergebnissen durchexerziert.
- Die Vorstellung zu den 4 KI-CLI-Plugin-Skills finden Sie im Abschnitt 3.1 Tooling und Infrastruktur.
- Für die Bewertung und Erkenntnisse dieser CAS-Arbeit bekommt die KI bei der Testgenerierung bewusst nur den reinen PHP-Code zu sehen — ohne PHPDoc oder Kommentare. Grund: So zeigt sich, was die KI rein aus dem Code selbst herausfinden kann, ohne zusätzliche Hilfestellung. (s. Abschnitt 1.1).

2.3 Kriterien zur Bewertung und erwartete Benefits

Kriterien zur Bewertung sind Kombination von 3 Testverfahren, nämlich (PHPUnit)Codeabdeckungstest, PhpStan-Test und Mutationstest.

Codeabdeckung (Code Coverage) beschreibt, wie viel Prozent des Quellcodes durch Tests ausgeführt werden. Eine hohe Codeabdeckung deutet auf eine gründlichere Testung und ein geringeres Fehlerrisiko hin. In meiner Bewertung wird in der ersten Linie **Funktions- und Methodenabdeckung berücksichtigt**, also wie viele Funktionen bzw. Methoden mindestens einmal durch Tests aufgerufen wurden. (<https://docs.phpunit.de/en/13.2/code-coverage.html>)

PHPStan ist ein leistungsstarkes Tool zur statischen Code-Analyse für PHP-Projekte. Es hilft Entwicklern, Fehler frühzeitig zu erkennen und die Codequalität zu verbessern, ohne den Code tatsächlich ausführen zu müssen. (<https://phpstan.org/>)

Ein **PHPStan Level** (<https://phpstan.org/user-guide/rule-levels>) bestimmt die Strenge der statischen Code-Analyse in PHPStan. . Die Level reichen von Stufe 0 (grundlegende Syntaxprüfungen) bis 10 (extrem strenge Typisierung). Bei meiner Bewertung ist der Level 6 in Einsatz. Level 6 ist der pragmatische Mittelweg:

- 1) Erzwingt vollständige Typ-Annotationen — wichtig für lesbare, wartbare Tests.
- 2) Die PHPStan-PHPUnit-Extension greift auf Level 6 optimal: Sie erkennt falsch verwendete Mocks, fehlerhafte Assertion-Signaturen und ungültige DataProvider-Strukturen.
- 3) Etablierter Standard in der TYPO3-Community für Testcode-Analyse.

Mutationstest ist ein Softwaretest, wo künstliche Bugs (Mutationen) im Code produziert werden, um festzustellen, ob die vorhandenen Tests ausreichen, um diese künstlichen Fehler zu entdecken. Die Codeabdeckung reicht nicht immer aus, Mutationstests sind für Unit-Tests unerlässlich, da sie die tatsächliche Qualität und Aussagekraft Ihrer Tests überprüfen. Ich habe **Infection** im Einsatz, Infection ist eine der bekanntesten Mutation Testing Frameworks für PHP (<https://github.com/infection/infection>) Bei meiner Bewertung ist **MSI-of covered** (<https://infection.github.io/guide/#Covered-Code-Mutation-Score-Indicator>) berücksichtigt. Nämlich Mutation Score Indicator, der nur Mutanten in Code berücksichtigt, der tatsächlich von Tests ausgeführt wird.

Hypothesen als erwartete Benefits:

Ich habe einige grobe Hypothesen wie folgendes aufgelistet.

H1: KI reduziert die Erstellungszeit deutlich gegenüber der manuellen Vorgehensweise. Bei trivialen Klassen ist eine sehr kurze Generierungszeit zu erwarten, bei komplexeren Klassen ist etwas mehr Korrekturzeit einzuplanen.

H2: Die statische Fehlerrate (z.B. PHPStan-Fehler) in KI-generierten Testklassen ist niedrig bzw. mit manuell erstellten Tests vergleichbar — die KI produziert also nicht systematisch mehr syntaktische/statisch erkennbare Fehler als ein Mensch.

H3: KI-generierte Tests erreichen eine hohe Methods Coverage, auch bei komplexeren Format-ViewHelpern. (Exakte Schwellenwerte werden erst nach der Messung festgelegt.)

H4: Der Grossteil der generierten Tests läuft ohne manuelle Korrekturen durch. Einzelne Klassen, insbesondere solche mit TYPO3-Abhängigkeiten, können Nacharbeit erfordern.

H5: Bei Glue-Code mit TYPO3-Dependencies erfordert das KI-generierte Mocking manuelle Korrekturen, ist aber nicht grundsätzlich falsch. Die KI liefert einen brauchbaren Ausgangspunkt, der genaue Aufwand wird im Messprotokoll erfasst.

H6: Unabhängig davon, ob zusätzlich PHPDoc und Inline-Kommentare als Kontext mitgegeben werden, liefert die KI generell Tests mit guter Assertions-Qualität: Grenzwerte werden geprüft und Edge-Cases sinnvoll behandelt.

H7: KI-generierte Tests erzielen nach Korrektur einen hohen Mutation Score (MSI of Covered, gemessen mit Infection PHP).

Die Messmethodik sowie die genauen Zielwerte, Qualitätsreferenz etc. werden im Abschnitt 3.5 nach den ersten Messdurchläufen gemessen und festgelegt und die Benefits werden ausführlich erläutert.

2.4 Vorgehen zur Einführung und Validierung

Die fünf Klassen wurden gezielt so ausgewählt, dass sie ein breites Spektrum von Komplexität und TYPO3-Abhängigkeit abdecken:

- 1) **JsonDecodeViewHelper** (Classes/ViewHelpers/Format/JsonDecodeViewHelper.php): Reine PHP-Logik ohne TYPO3-Kern-Abhängigkeit. Enthält relevante Edge-Cases (ungültiges JSON, leere Eingaben) und erfordert einfache Stubs/Mocks.
- 2) **CleanHtmlViewHelper** (Classes/ViewHelpers/Format/CleanHtmlViewHelper.php): Ebenfalls reine PHP-Logik ohne TYPO3-Abhängigkeiten. Der Einsatz umfangreicher regulärer Ausdrücke erhöht die Komplexität und macht Edge-Case-Tests besonders relevant.
- 3) **RoundViewHelper** (Classes/ViewHelpers/Format/RoundViewHelper.php): Reine PHP-Logik mit mathematischen Berechnungen (Rundung, Genauigkeit). Edge-Cases wie Grenzwerte und Gleitkommawerte erfordern sorgfältige Testabdeckung.
- 4) **Greeter** (Classes/Dummy/Greeter.php): Eine triviale Dummy-Klasse ohne TYPO3-Abhängigkeiten, Edge-Cases oder Mocking-Bedarf. Dient als Baseline und Kontrollfall für die einfachste Testsituation.
- 5) **DateViewHelper** (Classes/ViewHelpers/Format/DateViewHelper.php): Enthält reale TYPO3-Abhängigkeiten (Glue-Code). Das notwendige Mocking ist deutlich aufwändiger als bei den übrigen Klassen und zeigt exemplarisch die Grenzen der automatisierten Testgenerierung.

Begründung der Auswahl:

Die Klassen stammen alle aus packages/hf-view-helpers, da dort der Test-Audit-Workflow (Plugin typo3-test-audit) bereits eingerichtet ist und alle 47 Klassen klassifiziert vorliegen (4 Unit, 21 Edge-Cases, 18 Functional, 4 nicht testbar). Die gewählten fünf Klassen decken bewusst den Bogen von trivialer Logik über komplexe reine PHP-Logik bis hin zu TYPO3-abhängigem Glue-Code ab.

„Clue Codes“ ist eine Verschreibung von „Glue-Code“ (englisch für „Klebstoff-Code“). Der Begriff bezeichnet Code, der verschiedene Systemteile miteinander verbindet — in TYPO3 konkret: ViewHelper-Klassen, die TYPO3-interne Dienste aufrufen (z.B. `makeInstance()`, `CacheManager`, `DateUtility`). Solcher Code ist eng mit dem Framework verzahnt und schwer isoliert zu testen.

2.5 Bekannte Chancen und Risiken aus Praxis und Forschung

Bevor die eigenen Hypothesen (2.3) und das Validierungsvorgehen (2.4) im Detail festgelegt wurden, lohnt sich ein Blick auf bereits bekannte Erfahrungen — sowohl aus der Praxis als auch aus der Forschung. Sie bilden die Grundlage, auf der die eigenen Annahmen aufbauen.

Praxiserfahrungen zeigen ein Muster aus Vor- und Nachteilen, das sich in ähnlicher Form auch in dieser Arbeit wiederfindet: Zeitersparnis und schnelle Coverage-Gewinne stehen Risiken wie dem „Echo-Chamber“-Effekt, etc. — verbunden mit entsprechenden Massnahmen zur Risikominderung.

Akademische Forschung bestätigt dieses Bild ebenfalls weitgehend. Anders als eine Breitenstudie, die verschiedenste KI-Techniken über die Literatur hinweg vergleicht, liefert meine vorliegende Arbeit jedoch tool- und technologiespezifische Evidenz für einen konkreten LLM-Workflow (Claude API) im TYPO3/PHPUnit-Kontext.

3 Umsetzung

(Zusammenfassung)

3.1 Tooling, Infrastruktur und Systemlandschaft

Für die Umsetzung dieser Arbeit kommen folgende Tools und Technologien zum Einsatz:

DDEV — Lokale Entwicklungsumgebung auf Docker-Basis, in der das TYPO3-Projekt betrieben wird (<https://ddev.com/>)

TYPO3 14.3 — Das CMS-Framework, für das die Extension entwickelt und getestet wird (<https://typo3.com/de/typo3-v14>)

PHP 8.3 — Laufzeitumgebung für alle PHP-Klassen und Tests

PHPUnit 12 — Test-Framework für die Ausführung der Unit-Tests

TYPO3 TestingFramework — Erweitert PHPUnit um TYPO3-spezifische Hilfsfunktionen (beinhaltet in TYPO3 14.3)

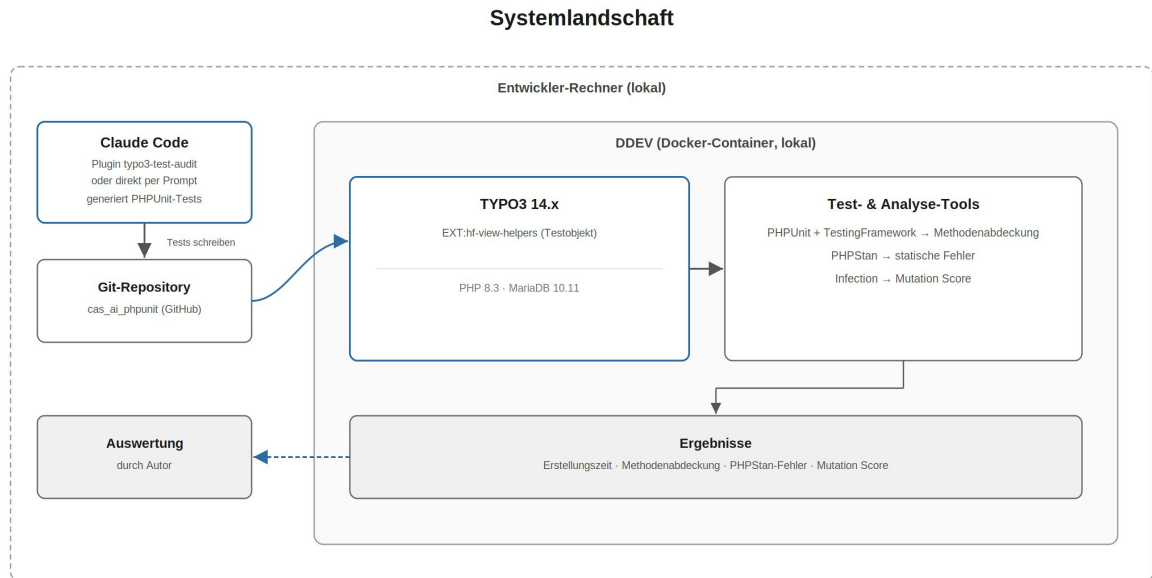
Claude API (claude-sonnet-4-6) — Das verwendete LLM zur Testgenerierung

Xdebug / php-code-coverage — Werkzeuge zur Messung der Codeabdeckung (<https://xdebug.org/>)

PHPStan — Statisches Analyse-Tool; prüft den PHP-Code auf Typfehler und potenzielle Bugs, ohne ihn auszuführen — wird hier eingesetzt, um die Qualität der generierten Testklassen zu bewerten (<https://phpstan.org/>)

Infection — Mutations-Test-Framework für PHP; prüft die Qualität der Tests, indem es den Quellcode gezielt verändert und überprüft, ob die Tests diese Änderungen erkennen (sogenannte Mutanten "töten") (<https://infection.github.io/>)

Abbildung als Systemlandschaft



Claude Code generiert — mithilfe des Plugins 'typo3-test-audit' oder direkt per Prompt — aus dem Quellcode der Extension PHPUnit-Testklassen und schreibt sie ins lokale Git-Repository 'cas_ai_phpunit' (auf GitHub versioniert).

Dieses Repository ist zugleich das Projektverzeichnis, das DDEV als Docker-Container auf demselben Rechner einbindet.

Innerhalb von DDEV laufen die TYPO3-14.x-Instanz mit dem Testobjekt 'hf-view-helpers' sowie die Analyse-Tools PHPUnit, PHPStan und Infection, die zusammen mit der gemessenen Erstellungszeit die vier Kennzahlen dieser Arbeit liefern.

Diese Ergebnisse werden anschliessend von mir als Autor ausgewertet.

3.2 Entwickeltes Resultat: Claude Code CLI Plugin

Das CLI-Plugin 'typo3-test-audit' wurde im Rahmen dieser Arbeit entwickelt und besteht aus vier aufeinander aufbauenden Skills, die direkt im Claude Code CLI aufgerufen werden:

/test-audit-text — Der erste Schritt: Das Plugin durchsucht alle PHP-Klassen einer Extension und klassifiziert sie automatisch in vier Kategorien: geeignet für Unit-Tests, Edge-Fälle (sowohl Unit- als auch Functional-Tests möglich), nur für Functional-Tests geeignet, oder nicht direkt testbar. Die Klassifizierung basiert auf Signalen im Quellcode, z. B. ob eine Klasse TYPO3-Infrastruktur wie CacheManager, ConnectionPool oder \$GLOBALS['TSFE'] verwendet. Das Ergebnis wird als .md- und .txt-Datei gespeichert. Auf diesem Report basiert auch die Auswahl der fünf Beispiellassen in dieser Arbeit.

/test-audit-chart — Liest den generierten Report und erstellt daraus ein SVG-Donut-Diagramm als visuelle Übersicht der Klassenverteilung.

/generate-unit-tests — Liest den .txt-Report und generiert automatisch PHPUnit-Testklassen für alle Unit- und Edge-Klassen. Pro Methode werden Happy-Path, Grenzwerte, Bool-Flags und Fehlerpfade abgedeckt. Alle Tests folgen dem AAA-Pattern (Arrange, Act, Assert). Existiert bereits eine Testdatei, analysiert der Skill vorhandene Kommentare und PHPDoc-Hinweise und ergänzt gezielt die fehlenden Testfälle — bestehender Code wird dabei nicht verändert.

/fix-unit-tests — Liest einen Infection-Report (Mutationstest) und ergänzt gezielt neue Testmethoden für alle überlebten Mutanten. So können Lücken in der Testabdeckung systematisch geschlossen werden.

Das Plugin hat eigentlich keinen direkten Bezug auf der Arbeit, das Plugin ist wiederverwendbar und nicht auf dieses Projekt beschränkt — es kann bei beliebigen TYPO3-Extensions eingesetzt werden, was ich sicherlich als einen Mehrwert und Benefit halte, möchte daher expliziert in diesem Abschnitt aufnehmen und kurz erläutern.

3.3 Nutzung von Plugin

Load Plugin

```
claude --plugin-dir .claude/plugins/typo3-test-audit
```

```
zhangsinian@SinianThinkPad:~/typo3-projects/cas_ai_phpunit$ claude --plugin-dir .claude/plugins/typo3-test-audit

Fable 5 is back.
Until July 7, you can use up to 50% of your plan's weekly usage limit on Fable 5. If you hit your limit, you can

> /typo3
/fix-unit-tests      (typo3-test-audit) Reads the infection-report.txt for a TYPO3 extension and adds target
the audit file is present). Use when the user asks to "fix unit tests", "kill escaped
/test-audit-text     (typo3-test-audit) Analyzes all PHP classes in a TYPO3 extension and produces a balance
per class. Use this skill when the user asks about "test audit", "test coverage overv
/test-audit-chart    (typo3-test-audit) Generates a standalone SVG donut chart from an existing test-audit
"draw chart" for a previously run test audit.
/generate-unit-tests (typo3-test-audit) Reads the test-audit-*.txt report for a TYPO3 extension and proces
(TODO, markTestIncomplete, etc.) and adds the missing test cases if any are found; ot
```

Run 4 Skills

```
/typo3-test-audit:test-audit-text <extension-path-or-name>
```

```
/typo3-test-audit:test-audit-chart <extension-path-or-name>
```

```
/typo3-test-audit:generate-unit-tests <extension-path-or-name>
```

```
/typo3-test-audit:fix-unit-tests <extension-path-or-name>
```

```
zhangsinian@SinianThinkPad:~/typo3-projects/cas_ai_phpunit$ claude --plugin-dir .claude/plugins/typo3-test-audit
Sonnet 4.6 with medium effort · Claude Pro
~/typo3-projects/cas_ai_phpunit

Fable 5 is back.
Until July 7, you can use up to 50% of your plan's weekly usage limit on Fable 5. If you hit your limit, you can
on Fable 5 with usage credits. Fable 5 draws down usage faster than Opus 4.8. Learn more

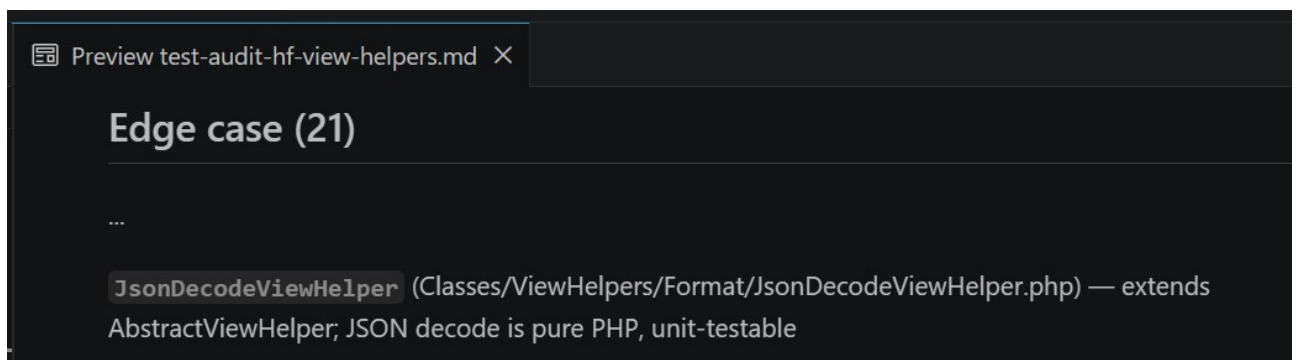
> /typo3-test-audit:test-audit-text <extension-path-or-name>
```


3.4 Beispielbasierte Demonstration

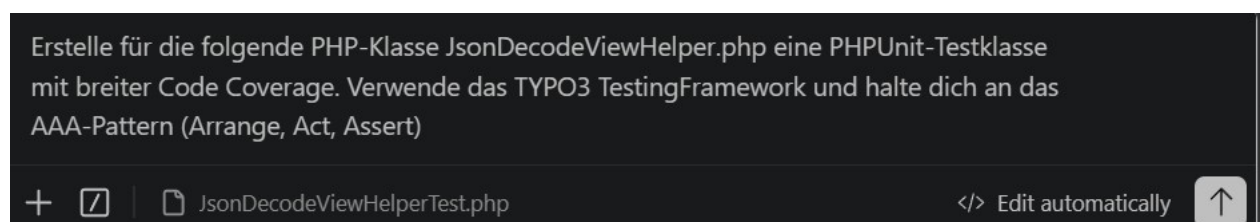
Dieser Abschnitt zeigt den kompletten Workflow ausführlich an einem konkreten Beispiel 'Classes/ViewHelpers/Format/JsonDecodeViewHelper.php': reine PHP-Logik ohne TYPO3-Kern-Abhängigkeit, — überschaubar und auch komplex genug, um Edge-Cases sowie einen Mutationstest zu zeigen.

Schritt 1: Klassifizierung durch den Skill '/test-audit-text'

Der Skill durchsucht alle Klassen der Extension 'hf-view-helpers' und ordnet 'JsonDecodeViewHelper' automatisch als unit-testbar ein, mit kurzer Begründung im Report (test-audit-hf-view-helpers.md):



Schritt 2: Prompt an die KI (während der Skill '/generate-unit-tests' generiert bzw. passt alle Klassen an)



Dabei wird eine Klasse automatisch generiert: (Inhalt der Klasse wird hier nicht angezeigt)
'packages/hf-view-helpers/Tests/Unit/ViewHelpers/Format/JsonDecodeViewHelperTest.php'

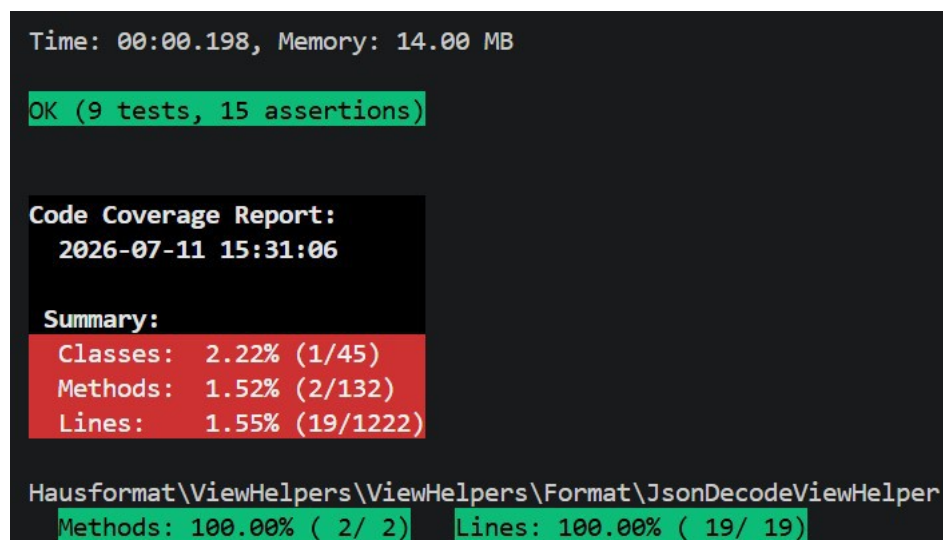
Schritt 3: PHPUnit-Test (mit Coverage) und PHPStan

PHPUnit

```
?> ddev exec php vendor/bin/phpunit \  
-c packages/hf-view-helpers/Build/phpunit/UnitTests.xml \  
packages/hf-view-helpers/Tests/Unit/ViewHelpers/Format/JsonDecodeViewHelperTest.php
```

PHPUnit - Code Coverage

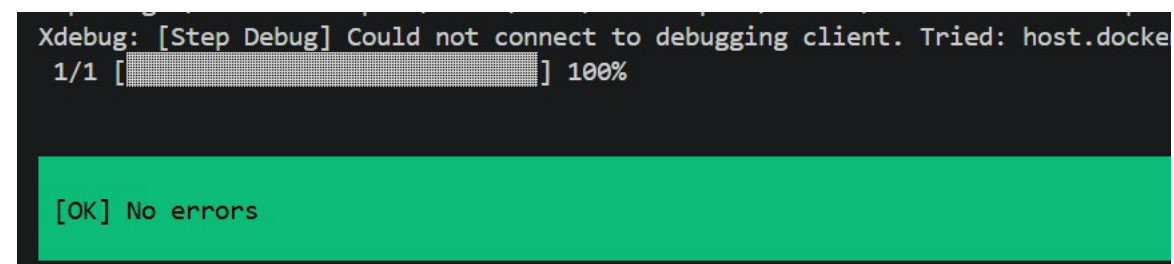
```
?> ddev exec XDEBUG_MODE=coverage php vendor/bin/phpunit \  
-c packages/hf-view-helpers/Build/phpunit/UnitTests.xml \  
--coverage-text \  
packages/hf-view-helpers/Tests/Unit/ViewHelpers/Format/JsonDecodeViewHelperTest.php
```



```
Time: 00:00.198, Memory: 14.00 MB  
  
OK (9 tests, 15 assertions)  
  
Code Coverage Report:  
2026-07-11 15:31:06  
  
Summary:  
Classes: 2.22% (1/45)  
Methods: 1.52% (2/132)  
Lines: 1.55% (19/1222)  
  
Hausformat\ViewHelpers\ViewHelpers\Format\JsonDecodeViewHelper  
Methods: 100.00% ( 2/ 2) Lines: 100.00% ( 19/ 19)
```

PHPStan

```
?> ddev php vendor/bin/phpstan analyse \  
-c packages/hf-view-helpers/Build/phpstan/phpstan.tests.neon \  
packages/hf-view-helpers/Tests/Unit/ViewHelpers/Format/JsonDecodeViewHelperTest.php
```

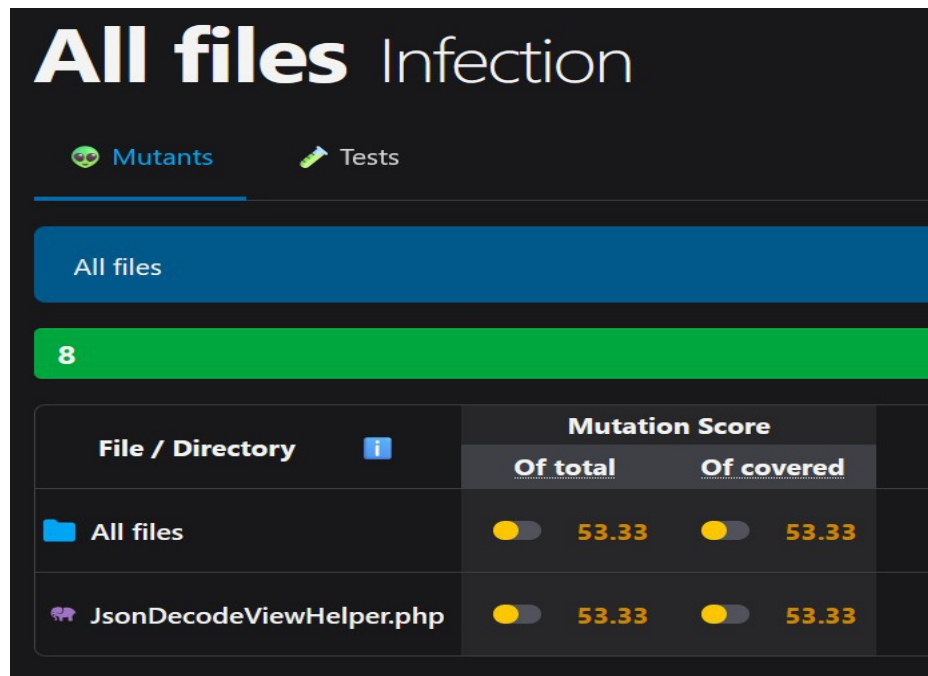


```
Xdebug: [Step Debug] Could not connect to debugging client. Tried: host.docker  
1/1 [████████████████████████████████████████] 100%  
  
[OK] No errors
```

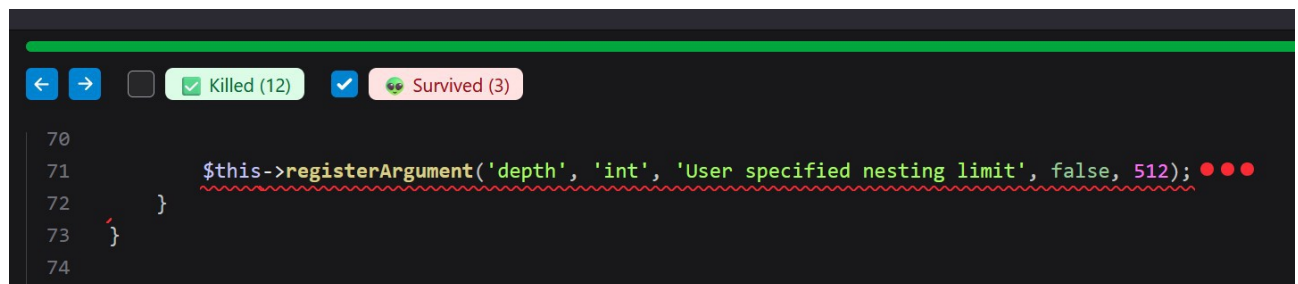
Die Ergebnisse sind alle GRÜN, alle UnitTests (9 tests, 15 insertions) sind bestanden, keine statischen Fehler sind findbar, Coverage sind 100% sowohl bei Methods auch bei Lines.

Schritt 4: Mutations-Test durch Infection

```
?> ddev exec php vendor/bin/infection run \
--configuration=packages/hf-view-helpers/Build/infection/infection.json5 --threads=4 \
--filter packages/hf-view-helpers/Classes/ViewHelpers/Format/CaseViewHelper.php
```



Wie bereits erwähnt, ist der Mutationstest – insbesondere der Wert MSI: Of Covered – ein wichtiger Indikator für die Qualität von UnitTests. In diesem Fall 53.33% wäre nicht zufrieden.



Der Befehl generiert Reports in verschiedenen Format, z.B. HTML-Datei mit besserer Darstellung (wie oben) und Text-Datei mit genaueren Angaben zu den Mutationen, also welche Mutationen (PHP Codes) wurden generiert und dabei survived. Hier wird ein survived Beispiel in HTML gezeigt, also die Definition eines Arguments „depth“ (Typ:int, Obligatorisch:false, Defaultwert:512)

Schritt 5: UnitTestCases erweitern bzw. fixen durch den Skill '/fix-unit-tests'

• All 12 tests pass (3 new).

Fix Unit Tests – Results

Source file	Mutants targeted	Test file	Status
Classes/ViewHelpers/Format/JsonDecodeViewHelper.php	7 (MethodCallRemoval x3, FalseValue x2, IncrementInteger x1, DecrementInteger x1)	Tests/Unit/ViewHelpers/Format/JsonDecodeViewHelperTest.php	tests pass

All 7 escaped mutants originated from initializeArguments()/registerArgument() calls, which the existing render()-based tests couldn't kill (they set arguments explicitly via setArguments(), bypassing the definitions). Added 3 new test methods that call prepareArguments() on a bare JsonDecodeViewHelper instance and assert each argument's type, isRequired(), and getDefaultValue() – covering json, assoc, and depth.

Alternativ soll der Fix per Prompt durchaus gleich wirkend sein, da ich bereits den Skill zur Verfügung habe, lasse ich den Skill durchlaufen.

Der Skill liest den Report, nämlich genau die Fälle, wo Mutationstests survibed sind, und erweitert die TestCases. Dabei werden 3 neue TestCases generiert, darunter ein TestCase bezüglich des Defaultwertes für den Arguemnt 'depth'.

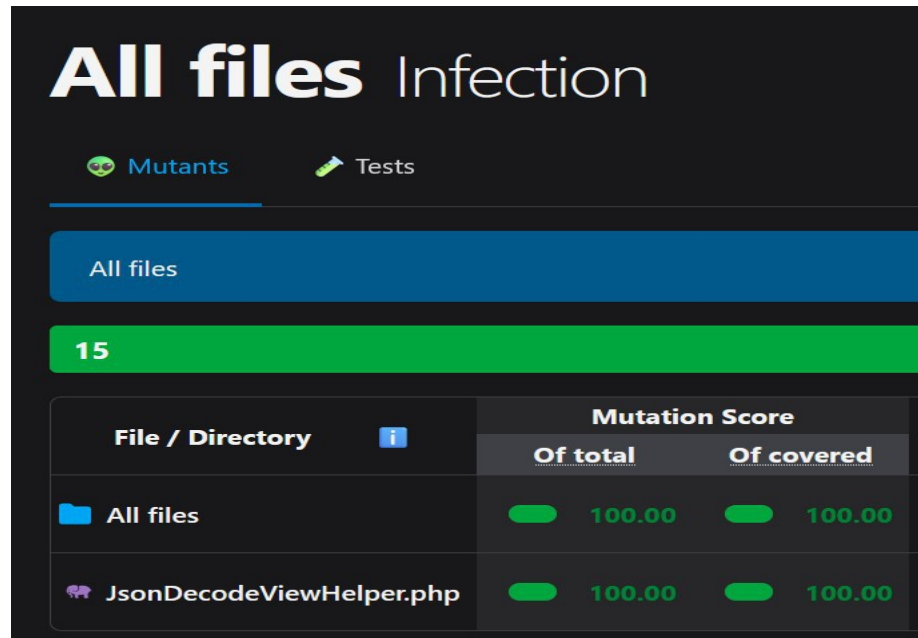
```

164+
165+     #[Test]
166+     public function prepareArgumentsRegistersDepthAsOptionalIntDefaultingTo512(): void
167+     {
168+         $subject = new JsonDecodeViewHelper();
169+         $definitions = $subject->prepareArguments();
170+         self::assertArrayHasKey('depth', $definitions);
171+
172+         self::assertSame('int', $definitions['depth']->getType());
173+         self::assertFalse($definitions['depth']->isRequired());
174+         self::assertSame(512, $definitions['depth']->getDefaultValue());
175+     }
176+ }
177

```

Es ist ein klassischer Fall, PHPUnit hat zuvor z.B. den Defaulverhalten nämlich den Defaultwert 512 nicht richtig getestet, also der Mutationstest ändert den Wert 512 zu 511 oder 513 (in der originalen Klasse, Zeile 71), läuft die UnitTests immer mit Erfolg durch. Durch den neuen TestCase wird der Fall abgedeckt, also nun ändert Mutationstest den Wert zu 511, bei Aufruf der Funktion ohne Argument-'depth', bekommt dann die Funtion den Argument mit Wert 511, 511 ist nicht gleich 512, wird der Test dann als killed eingestuft.

Schritt 6: Kontrolle mit Mutationstest nach dem Fix



Nach dem Fix zeigt dann der Mutationstest für die betroffene Klasse 100% GRÜN.

3.5 Pilotnutzung

Abschnitt 3.4 demonstriert den vollständigen Workflow aber exemplarisch an einer einzelnen Klasse (JsonDecodeViewHelper), wurde derselbe Ablauf im Rahmen eines Pilotlaufs auf die übrigen vier ausgewählten Klassen angewendet, nämlich ForViewHelper, RoundViewHelper, Greeter und DateViewHelper. Es kamen dieselben Vorgehensweise sowie Skills und Tools (Plugin 'typo3-test-audit', PHPUnit, PHPStan, Infection) zum Einsatz.

Ziel dieses Pilotlaufs war zu prüfen, ob der Workflow auch bei unterschiedlicher Klassenkomplexität und unterschiedlichem TYPO3-Abhängigkeitsgrad reproduzierbar funktioniert, danach geht zur systematischen Auswertung in Abschnitt 3.6.

Qualitative Beobachtungen aus dem Pilotlauf:

- Greeter, einfachste Dummy-/Testklasse ohne TYPO3-Abhängigkeiten, lief bereits im ersten Anlauf vollständig fehlerfrei durch (First-Run GREEN) — kein Korrekturbedarf.
- Bei den reinen PHP-Logik-Klassen mit höherer Komplexität, ForViewHelper, RoundViewHelper, traten im ersten Wurf vereinzelt PHPStan-Fehler auf. Diese liessen sich in der Korrekturphase per KI in wenigen Minuten beheben.
- DateViewHelper als einzige Klasse mit echten TYPO3-Dependencies (Glue-Code) zeigte erwartungsgemäss den grössten Korrekturbedarf.

Der Pilotlauf bestätigt damit, dass der in 3.4 gezeigte Einzelfall kein Sonderfall war, sondern sich auf Klassen unterschiedlicher Komplexität und Abhängigkeitstiefe übertragen lässt. Die generierte Testklasse mussten nicht verworfen oder von Grund auf neu geschrieben werden — die KI lieferte in jedem Fall einen sinnvollen, weiterverwendbaren Ausgangspunkt.

Die vollständigen quantitativen Ergebnisse dieses Pilotlaufs — Erstellungszeit, Methods Coverage, Mutation Score, PHPStan-Fehler und Assertions-Qualität je Klasse — werden in Abschnitt 3.6 tabellarisch dargestellt, die Hypothesen aus Abschnitt 2.3 werden einbezogen.

3.6 Messung und Beobachtung der Benefits und Effekte

Messmethodik

Für jede der fünf Klassen wird das Messprotokoll in zwei Gruppen ausgefüllt: einmal für die manuelle Referenz, einmal für die KI-gestützte Variante.

Manuell: Die Stoppuhr startet zu Beginn der Test-Erstellung. Der Entwickler liest die Zielklasse, versteht die öffentlichen Methoden, überlegt sinnvolle TestCases und schreibt die Testklasse vollständig aus. Die Messung gilt als abgeschlossen, sobald zwei Kriterien erfüllt sind: PHPUnit läuft fehlerfrei durch, und PHPStan meldet keine Fehler. Erst dann wird die Stoppuhr gestoppt.

KI: Die Zeitmessung erfolgt in zwei Phasen:

Phase 1 — Generierung: Die Stoppuhr startet mit dem Absenden des Prompts. Die KI legt die fehlende Testklasse samt Testmethoden an. Die Stoppuhr stoppt, sobald die Generierung abgeschlossen ist.

Der Prompt auf Deutsch lautet z.B.: Erstelle für die folgende PHP-Klasse

JsonDecodeViewHelper.php eine PHPUnit-Testklasse mit breiter Code Coverage. Verwende das TYPO3 TestingFramework und halte dich an das AAA-Pattern (Arrange, Act, Assert).

Phase 2 — Korrektur: Die Stoppuhr startet erneut. Die generierte Testklasse wird mit PHPStan analysiert; auftretende Fehler werden per KI korrigiert. Die Stoppuhr stoppt, sobald PHPStan keine Fehler mehr meldet.

Auch für die KI-Variante gilt dasselbe Abschluss-Kriterium wie für die manuelle Messung: PHPUnit läuft fehlerfrei, PHPStan meldet keine Fehler.

Bewertungsskala Assertions (Stufe: 1–5)

(s. 2.3 Kriterien zur Bewertung und erwartete Benefits)

Die Stufenzuordnung basiert grundsätzlich auf subjektiver Beobachtung sowie den folgenden Überlegungen und Kriterien.

Stufe-5: Alle öffentlichen Methoden geprüft, Assertions für Happy-Path, Grenzwerte und Fehlerfälle, keine Tautologien, Testwerte unterscheiden ähnliche Implementierungen, Mocking korrekt (richtiger Typ createMock/createStub, richtige Methode, kein unnötiges Mocking), PHPStan-Errors: 0–1, MSI > 90%

Stufe-4: Grundsätzlich korrekte Assertions ohne Tautologien, einzelne Grenzwerte oder Nebenmethoden fehlen, Mutationstests mehrheitlich bestanden, Mocking weitgehend korrekt, kleinere Typ-Ungenauigkeiten, PHPStan-Errors: 2–3, MSI 75–90%

Stufe-3: Assertions vorhanden, aber mit strukturellen Schwächen: Testwerte unterscheiden ähnliche Implementierungen nicht, und/oder ganze Methoden oder Argument-Defaults sind vollständig ungeprüft, Mocking zum Teil strukturell falsch (falscher Typ, fehlende oder überflüssige Abhängigkeiten), PHPStan-Errors: 5–10, MSI 50–75%

Stufe-2: Mehrere tautologische oder triviale Assertions (z.B. `assertInstanceOf`, `assertNotNull`), Kernlogik wird nicht wirklich verifiziert, Mocking fehlend oder sinnlos, PHPStan-Errors: > 10, MSI < 50%

Stufe-1: Tests bestehen formal, prüfen aber keine sinnvollen Eigenschaften, Implementierung ist falsch oder fehlerhaft, oder jede Implementierung würde die Tests bestehen, sehr viele PHPStan-Errors: > 10, sehr geringer MSI < 50%

Ergebnistabelle (2 Gruppen: Manuell und KI)

(?: Zeitaufwand deutlich über 60 Minuten; keine Messung, daher nicht erfasst.)

Klasse	Zeit Man.	Coverage Man. (Methods)	Mutationstest Man. (MSI of Covered)	PHPStan Man.	
JsonDecodeViewHelper	> 30 Min.	50%	41%	0	
ForViewHelper	30 Min.	100%	53%	0	
RoundViewHelper	> 60 Min.	(?)	(?)	(?)	
Greeter	30 Min.	100%	100%	0	
DateViewHelper	> 30 Min.	50%	48%	0	
Klasse	Zeit KI (Error-Fix)	Coverage KI (Methods)	Mutationstest KI vor und nach dem Fix (MSI of Covered)	PHPStan KI (Error vor und nach dem Fix)	Assert.-Stufe KI vor und nach dem Fix (1–5)
JsonDecodeViewHelper	1.5 Min.	100%	53% -> 100%	0	3 -> 5
ForViewHelper	3 Min. (+2 Min.)	100%	61% -> 100%	6 -> 0	3 -> 5
RoundViewHelper	2.5 Min. (+1 Min.)	100%	82% -> 100%	1 -> 0	4 -> 5
Greeter	< 1 Min.	100%	100%	0	5
DateViewHelper	2.5 Min. (+1 Min.)	100%	57% -> 100%	2 -> 0	4 -> 5

Bezug zu den Hypothesen

Die Hypothesen vom Abschnitt 2.3 können zum grossen Teil bestätigt und erzielt werden, ich erläutere sie ausführlich wie folgendes.

H1 — Zeitersparnis: Die manuelle Erstellung dauerte je nach Klasse zwischen 30 und über 60 Minuten — wobei RoundViewHelper mit über 60 Minuten den Höchstwert markiert. Die KI-gestützte Generierung inkl. Korrekturzeit liegt bei allen fünf Klassen unter 5 Minuten: Greeter unter 1 Minute, JsonDecodeViewHelper 1,5 Minuten, RoundViewHelper und DateViewHelper je 3,5 Minuten, ForViewHelper als aufwändigster Fall bei 5 Minuten total. Die Zeitersparnis ist damit in allen Fällen deutlich — auch bei komplexeren Klassen beträgt der Faktor mindestens 6. H1 wird durch die Messdaten klar bestätigt.

H2 — Statische Fehlerrate (PHPStan): bezüglich der Daten in der Ergebnistabelle waren die manuell erstellten Tests durchgehend PHPStan-fehlerfrei. Bei den KI-generierten Tests zeigte sich vor der Korrekturphase ein anderes Bild. Die KI produziert im ersten Wurf also nicht weniger, sondern bei einem Teil der Klassen sogar mehr statisch erkennbare Fehler als ein Mensch. H2 wird durch die Messung damit nicht ganz bestätigt. Bemerkenswert ist jedoch, wie schnell sich diese Fehler beheben lassen. Die KI ist also zumindest in der Lage, statische Fehler sehr rasch selbst zu korrigieren.

H3 — Methods Coverage: Die manuellen Tests erreichen je nach Klasse 50–100% Methods Coverage; bei JsonDecodeViewHelper und DateViewHelper nur 50%, weil nicht alle öffentlichen Methoden abgedeckt wurden. Die KI-generierten Tests erreichen bei allen fünf Klassen 100% Methods Coverage — unabhängig von der Komplexität der Klasse. H3 wird vollständig bestätigt und übertrifft die ursprüngliche Erwartung.

H4 — Validität (First-Run GREEN): Als „First-Run GREEN“ gilt eine Testklasse, wenn PHPUnit fehlerfrei läuft und PHPStan keine Fehler meldet. Nach diesem strengen Kriterium erfüllen zwei von fünf Klassen (JsonDecodeViewHelper und Greeter) diese Bedingung sofort. ForViewHelper (6 PHPStan-Fehler), RoundViewHelper (1 Fehler) und DateViewHelper (2 Fehler) benötigten eine Korrekturphase. Die Quote liegt damit bei 40% — unterhalb der hypothetisierten Mehrheit. Der Korrekturbedarf war jedoch in allen Fällen gering und in unter 2 Minuten behoben; ein

vollständiges Scheitern trat nie auf. H4 wird in der strengen Definition nicht bestätigt, relativiert sich aber durch den niedrigen Korrekturaufwand.

H5 — Mocking-Korrektheit bei Glue-Code: Der DateViewHelper als einzige Klasse mit echten TYPO3-Dependencies lieferte einen direkt brauchbaren Ausgangspunkt: Assertions-Stufe 4 vor dem Fix, 2 PHPStan-Fehler, MSI 57%. Nach kurzer Korrektur: Stufe 5, PHPStan-fehlerfrei, MSI 100%. Das Mocking war strukturell nicht grundsätzlich falsch — es fehlten lediglich korrekte Typ-Annotationen. H5 wird bestätigt: Die KI scheitert bei Glue-Code nicht, liefert aber einen Entwurf, der gezielter Nacharbeit bedarf.

H6 — Assertions-Qualität: Die KI liefert generell Tests mit guter Assertions-Qualität: Grenzwerte werden geprüft, Edge-Cases sinnvoll behandelt. Die Ergebnistabelle zeigt, dass die initiale Assertions-Stufe bei einfacheren Klassen (Greeter, RoundViewHelper, DateViewHelper) bereits bei 4–5 lag; bei komplexeren Klassen mit mehr Methoden (JsonDecodeViewHelper, ForViewHelper) startete die KI auf Stufe 3, erreichte aber nach gezielter Nachbesserung durchgehend Stufe 5. Das bedeutet: Die KI erzeugt keine trivialen oder tautologischen Tests — sie prüft die Kernlogik, erkennt relevante Eingabewerte und deckt Fehlerfälle ab. H6 wird bestätigt.

H7 — Mutationstest (MSI of Covered): Vor Korrekturen lagen die MSI-Werte zwischen 53% (JsonDecodeViewHelper) und 100% (Greeter). Nach Korrekturen erreichten alle fünf Klassen 100% MSI. Selbst der schwächste Ausgangswert (53%) konnte durch gezielte Assertions-Verbesserungen auf 100% angehoben werden. H7 wird vollständig bestätigt: KI-generierte Tests erzielen nach Korrektur einen sehr hohen Mutation Score und sind damit nicht nur syntaktisch, sondern auch inhaltlich aussagekräftig.

4 Diskussion

(Zusammenfassung)

4.1 Erreichte Ergebnisse und Beitrag der Arbeit

Die Messungen aus Abschnitt 3.6 bestätigen den Kern der Arbeit: KI-gestützte Testgenerierung ist unter TYPO3 14 praktisch einsetzbar und reduziert den Zeitaufwand drastisch, ohne die Testqualität systematisch zu verschlechtern.

Die meisten Hypothesen wurden bestätigt: u.a. Zeitersparnis, viel breitere Methods-Coverage bzw. nach der Korrektur gegenüber manuelle Referenz, Stube/Mocking bei Glue-Code überwiegend brauchbar, gute Assertion-Qualität, etc. Nicht bzw. nur teilweise bestätigt wurden eine Hypothese, nämlich: die KI produzierte im ersten Wurf mit teils mehr PHPStan-Fehler als die manuelle Referenz, die Korrekturphase kurz aber benötigt.

Der zentrale Befund liegt damit nicht in "KI ersetzt den Menschen", sondern "KI unterstützt den Menschen": Die KI übernimmt die zeitintensive Fleissarbeit der Testerstellung praktisch vollständig, während eine kurze, gezielte menschliche (oder KI-gestützte) Korrekturphase notwendig bleibt, bzw. mit Hilfe von Inline-Kommentar als Hinweis zum konkreten TestCase bringt KI theoretisch viel bessere Qualität. Diese wohl benötigte Korrektur kostet wenige Zeit — der Gesamtaufwand bleibt damit trotzdem um ein Vielfaches unter der manuellen Referenz.

Reine Logik vs. Glue-Code: Bei den drei reinen PHP-Logik-Klassen (JsonDecodeViewHelper, RoundViewHelper sowie der Dummy Greeter) lieferte die KI durchgehend hochwertige, teils sofort fehlerfreie Tests. Beim einzigen Glue-Code-Fall mit echten TYPO3-Abhängigkeiten (DateViewHelper) war das Ergebnis vor Korrektur bereits solide, verbesserte sich aber erst durch gezielte Nachbesserung.

Beitrag der Arbeit:

- Praktischer Nachweis, dass KI-gestützte PHPUnit-Testgenerierung unter TYPO3 14 funktioniert und unter Umständen produktionsnahe Ergebnisse liefert.
- Ein wiederverwendbares, projektunabhängiges Werkzeug (Plugin typo3-test-audit), das über diese Arbeit hinaus im Tagesgeschäft einsetzbar ist.
- Eine differenzierte Empfehlung: KI-Generierung eignet sich besser für reine Logik-Klassen; bei TYPO3-Glue-Code liefert sie einen brauchbaren, aber korrekturbedürftigen Ausgangspunkt.

4.2 Bezug zu bestehenden Erfahrungen und Studien

Vergleich mit anderen Praxiserfahrungen

Praxisberichte oder Referenzen zum Einsatz generativer KI (z. B. GitHub Copilot, JetBrains AI, ChatGPT) im Software-Testing findet man im Internet oder Publikationen, sie zeigen, zusammengefasst, ein klares und ähnliches Muster aus Vorteilen und Grenzen.

Vorteile: KI spart Zeit bei repetitivem, wiederkehrendem Code und liefert schnell ein brauchbares Testgerüst. Die Code-Coverage steigt dadurch rasch an. Zudem liefert KI auf Nachfrage schnell Ideen für Edge-Cases (z. B. Null-Werte, Extremwerte), die man selbst leicht übersieht.

Nachteile: Der „Echo Chamber“-Effekt, also KI übernimmt Fehler aus dem Code, statt sie zu erkennen — rechnet eine Funktion aus Versehen falsch, bestätigt die KI dieses falsche Ergebnis trotzdem als richtig. KI neigt dazu, Mocks exzessiv zu nutzen anstatt das tatsächliche funktionale Rückgabe-Verhalten einer Komponente zu prüfen eben auch veraltete Test-Methoden (z.B. deprecated PHPUnit-Features) zu nutzen.

Um dieses Risiko zu senken, braucht es drei Validierungsmassnahmen:

Menschliche Kontrolle: KI-generierter Testcode darf nie ungeprüft übernommen werden — er muss wie der Code eines unerfahrenen Entwicklers auf fachliche Richtigkeit und bezüglich der Geschäftsanforderungen geprüft werden.

Mutationstests (z. B. Infection PHP): Der eigentliche Härtetest. Ein Tool baut gezielt Fehler in den Produktionscode ein — schlägt der Test dabei nicht fehl, ist die Assertion nutzlos und muss überarbeitet werden.

Ergänzende Integrationstests: Da KI Abhängigkeiten (Datenbanken, APIs) meist einfach wegmockt, müssen wichtige Systempfade zusätzlich mit echten Integrationstests abgesichert werden.

Einordnung in die akademische Forschung

Meine Ergebnisse lassen sich in den breiteren Forschungsstand mehr oder weginer einordnen.

Durrani et al. (2025) untersuchen mittels eines Zero-Truncated-Poisson-Modells quantitativ, wie sich KI-Einsatz auf Genauigkeit und Effizienz über sechs SE-Phasen hinweg auswirkt. Für die Testing-Phase zeigen sie, dass KI die Test-Accuracy signifikant verbessert ($p < 0.0001$), der Effizienzgewinn jedoch statistisch nicht signifikant ist ($p = 0.1359$) — mit dem Hinweis, dass KI-generierte Testsuiten weiterhin manuelle Validierung bzw. "human-in-the-loop"-Aufsicht benötigen.

Mein Erkenntnis deckt sich mit den eigenen Messungen: Auch hier war bei 3 von 5 Klassen eine Korrekturphase nötig (H2/H3), und die Zeitersparnis (H1) war zwar sehr deutlich, ersetzte aber keine menschliche Kontrolle.

Vergleich mit einer Breitenstudie z.B. von Durrani et al., liefert meine Arbeit eher tool- und technologiespezifische Evidenz für einen konkreten LLM-Workflow (Claude API) im TYPO3/PHPUnit-Kontext.

4.3 Empfehlung, Innovation, Lernen

Empfehlung

Reine PHP-Logik-Klassen priorisieren: Für Klassen ohne TYPO3-Abhängigkeiten liefert die KI bereits im ersten Wurf hochwertige Tests, sollte im Tagesgeschäft eingeführt werden.

PHPDoc und Inline-Kommentare gezielt für die Testgenerierung nutzen: In dieser Arbeit liess ich die KI bewusst nur mit reinem Quellcode ohne Kommentare füttern lassen. Für den produktiven Einsatz empfehlungswert. — dieser zusätzliche Kontext dürfte die Trefferquote und Assertion-Qualität weiter erhöhen, dagegen erhöht sich ggf. der Aufwand. Ich rede von einer guten Balance zwischen Aufwand und Qualität.

Korrekturschritt fest einplanen, nicht überspringen: generierter Testcode darf nie ungeprüft übernommen werden, z.B. Typ-Annotationen und Mock-Konfiguration etc. sollen durch eine erfahrene Person geprüft werden, bevor die Testklasse in die CI/CD-Pipeline gelangt.

Mutationstests als Qualitätsgate etablieren: Da Codeabdeckung allein die Testqualität nicht zuverlässig misst, sollte Infection dauerhaft als zweite Prüfebene neben PHPUnit und PHPStan im Projekt verankert werden.

Test-Driven Development (TDD) mit KI-Unterstützung: TDD ist eine moderne, viel beachtete Methode, bei der zuerst der Test und erst danach der Code entsteht. Da die KI Tests in Minuten liefert, könnte künftig der Mensch den ersten Test schreiben und dann unter anderen die KI Implementierung, Refactoring und Edge-Cases, etc. übernehmen.

Innovation

Der Neuheitswert dieser Arbeit liegt weniger im theoretischen Nachweis, dass LLMs Unit-Tests generieren können — das ist in der Literatur bereits breit dokumentiert (siehe Durrani et al. 2025, Abschnitt 4.2) —, sondern in der konkreten, wiederverwendbaren Umsetzung im TYPO3-Ökosystem:

Das Plugin typo3-test-audit (Abschnitt 3.2) ist projektunabhängig konzipiert und kann ab sofort bei beliebigen TYPO3-Extensions eingesetzt werden.

Die systematische Klassifizierung von Testbarkeit (Unit / Edge / Functional / nicht testbar) automatisiert einen Schritt, der bislang rein erfahrungsbasiert und manuell erfolgte. Der Skill test-audit-text macht dieses Wissen für jede beliebige Extension reproduzierbar verfügbar.

Die Kombination aus drei unabhängigen Qualitätskennzahlen — Methods Coverage, PHPStan-Fehlerrate und Mutation Score (MSI of Covered) liefert eine deutlich robustere, objektivere Bewertungsgrundlage für KI-generierten Testcode.

Lernen

Die persönlichen Lernziele aus Abschnitt 1.3 habe ich durch die Arbeit erreicht. PHPUnit war vorher nicht Teil meines Alltags. Durch das manuelle Schreiben der Referenz-Tests und das Prüfen der KI-generierten Tests habe ich gelernt, wie ein sinnvoller Testfall aufgebaut ist (Happy-Path, Grenzwerte, Fehlerfälle) und wie man einen Coverage-Report richtig liest, statt nur auf die Prozentzahl zu schauen.

Besonders viel gelernt habe ich beim Mocking von TYPO3-Abhängigkeiten, am Beispiel des DateViewHelper (H5). Erst beim Beheben der Fehler in den KI-generierten Mocks wurde mir klar, welche Typ-Angaben und welche Mock-Konfiguration wirklich nötig sind. Dieses Wissen kann ich auch bei anderen TYPO3-Klassen im Alltag nutzen.

Auch der Begriff Mutation Score war mir vorher unbekannt. Durch den praktischen Einsatz von Infection und das Lesen der MSI-Werte (Abschnitt 3.6) verstehe ich jetzt, warum eine hohe Codeabdeckung allein noch nichts über die Qualität der Tests aussagt.

Zusätzlich habe ich mit Claude Code ein Werkzeug kennengelernt, das mehr kann als nur Code schreiben. Beim Bau eigener Skills (Abschnitt 3.2) habe ich gelernt, wie man ein KI-CLI-Tool gezielt für wiederkehrende Aufgaben einrichtet. Zusammen mit PHPStan ist das ein Werkzeugset, das ich auch nach dieser Arbeit im Berufsalltag weiter nutzen werde.

Literatur- und Quellenverzeichnis

- Durrani, U. et al. (2025). Impact of Artificial Intelligence on Software Engineering Phases and Activities (2013–2024): A Quantitative Analysis Using Zero-Truncated Poisson Model
- DDEV: <https://ddev.com/>
- TYPO3 14.3: <https://typo3.com/de/typo3-v14>
- PHPStan: <https://phpstan.org/>
- PHPStan Rule Levels: <https://phpstan.org/user-guide/rule-levels>
- Infection PHP: <https://infection.github.io/>
- Infection — Covered Code Mutation Score Indicator: <https://infection.github.io/guide/#Covered-Code-Mutation-Score-Indicator>
- Xdebug: <https://xdebug.org/>
- PHPUnit Code Coverage: <https://docs.phpunit.de/en/13.2/code-coverage.html>
- DORA — Value Stream Management: <https://dora.dev/guides/value-stream-management/>

Glossar

AAA-Pattern: Arrange/Act/Assert — Strukturprinzip für Testmethoden.

Glue-Code: Code, der verschiedene Systemteile miteinander verbindet — hier: ViewHelper-Klassen, die TYPO3-interne Dienste aufrufen (z. B. 'makeInstance()', 'CacheManager', 'DateUtility').

Methods Coverage: Anteil der öffentlichen Methoden, die mindestens einmal von einem Test aufgerufen werden.

MSI (of Covered): Mutation Score Indicator, gemessen mit Infection PHP; berücksichtigt nur Mutanten in tatsächlich von Tests ausgeführtem Code.

PHPStan-Level: Stufe der Strenge der statischen Code-Analyse (0 = grundlegend, 10 = extrem streng); in dieser Arbeit Level 6.

Mutationstest: Testverfahren, bei dem künstliche Fehler (Mutanten) in den Code eingebaut werden, um zu prüfen, ob bestehende Tests diese erkennen.

Abkürzungsverzeichnis

Abkürzung / Bedeutung

AAA	Arrange, Act, Assert (Struktur-Pattern für Testmethoden)
CAS	Certificate of Advanced Studies
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
CMS	Content Management System
DDEV	Docker-basierte lokale Entwicklungsumgebung
DI	Dependency Injection
DORA	DevOps Research and Assessment
EXT	Extension (TYPO3-Erweiterung)
FHNW	Fachhochschule Nordwestschweiz
GmbH	Gesellschaft mit beschränkter Haftung
GPT	Generative Pre-trained Transformer
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LLM	Large Language Model
MSI	Mutation Score Indicator
PHP	PHP: Hypertext Preprocessor
SVG	Scalable Vector Graphics
TDD	Test-Driven Development
TSFE	TYPO3 Frontend Controller (\$GLOBALS['TSFE'])
VSM	Value Stream Mapping (Wertstromanalyse)

(anpassen; Seitenzahlen werden nicht mitgezählt)